

Metropolis Nexus

Group 66

Tchakmakian, Karl
Parry-Wingfield, Jack
Grillet-Aubert, Alexis
Le, Khang

I. Requirement Analysis

1. Introduction

a. Purpose

Metropolis Nexus is a strategic platform designed to manage a national fleet of rental vehicles. The system addresses the problem inherent in multi-hub models, where cars tend to pile up in certain cities because customers rent them in one location and drop them off in another. This creates an imbalance where some branches have too many cars while others have none. The design is motivated by the need for cost-efficient fleet rebalancing, calculating whether the projected revenue of a target market justifies the specific **Inter-Area Transfer Fees** and the **Opportunity Cost** of removing assets from high-revenue, high-population hubs like Montreal or Toronto.

b. Scope and special requirements

The system's sphere of interest covers a national network divided into primary logistical units called Areas (e.g., Montreal Metro, GTA, Western Hubs). Within these areas, the system tracks individual Branches as the physical points of transaction and inventory storage. This allows the platform to model the financial impact of national supply-chain logistics and ensure that high-population areas like Montreal are never "starved" of inventory.

c. Terminology

Opportunity Cost: The potential revenue a source Area loses when a high-demand vehicle is relocated to a different territory.

Inventory Leveling: The process of rebalancing the fleet to minimize out-of-stock risk across branches.

Resource Saturation: The point at which a branch's labor (employees) or inventory (vehicles) can no longer meet local transaction volume.

d. Resources

Relocation Algorithm Logic: Inspired by [The static bike relocation problem with multiple vehicles and visits](#). This research provides the framework for rebalancing rental assets across a multi-hub network at minimum cost.

Optimization Framework: [Static bike relocation optimization applied to Montreal's BIXI bike sharing system](#). This paper provides the logistical logic for "Inventory Leveling" and calculating "Relocation Penalties" across regional boundaries.

2. Database description

a. Entities and their attributes

The following entities will be stored as entities of a relational database:

Vehicle: A vehicle is any mode of transportation that is available to rent. The database stores information for each vehicle, including its *VIN*, *license_plate*, *mileage*, *make*, *model*, and *status*.

The status attribute represents the vehicle's current rental status: Available, Rented, Relocating (in-transit).

Each vehicle is also uniquely identifiable by its VIN number, which serves a Vehicle's primary key.

VehicleClass: A VehicleClass represents a category of vehicles that share similar characteristics between each other.

The database stores the class *name*, the *security deposit* needed to rent vehicles from within that class, and finally the *classID*.

Branch: A branch is a rental location where vehicles are stored, picked up, and returned.

The database stores each branch with their unique *branchID*, *phone number*, *address*, and the city in which the branch is located.

Area: An area represents a geographical region that represents logistical hubs. The database stores an *AreaID* with the area name that it represents.

Employee: An employee represents a person that works for the rental agency. The database stores their *name*, *salary*, and *employee ID*.

Branch Manager: A branch manager is a sub-type of employee that inherits an employee ID. Each branch manager is responsible for managing a specific branch. BranchManager has no additional attributes beyond Employee.

- **Note:** During our design process, we considered removing the **Branch Manager** entity since it introduces no additional attributes beyond **Employee**, but we decided to keep it to explicitly enforce the constraint that a manager uniquely manages a branch.

Customer: A customer is a person who makes reservations and enters rental agreements.

The database stores each customer's *email*, *name*, *address*, and *license_expiry* of their driving license. Each customer is uniquely identifiable by their email address and their *license_expiry* allows our system to ensure each driver has a valid legal permit.

Reservation: A reservation represents a booking request made by a customer indicating their intent to rent a vehicle. The database stores each reservation with a unique reservation identifier (*resID*) and the timestamp at which the booking was created (*bookedAtTime*).

A reservation must be associated with exactly one customer and may reserve one or more rental periods. A reservation does not immediately constitute a legal contract, it is only fulfilled into an agreement at the moment of vehicle pickup.

Agreement: An agreement is a legal contract created whenever a customer rents a car.

The database stores a unique *contract_id* with the specific *planType* chosen and *totalCost*.

A plan type represents the specific rental plan chosen: Daily, Weekly, Monthly, etc.

- **Note :** A Reservation may exist without an Agreement, but an Agreement **must** originate from exactly one Reservation.

RentalPeriod: A rentalPeriod is a weak entity of reservation that represents a scheduled rental time. Each rental period is uniquely represented by a combination of *resID* and *periodID*. Additionally, the database stores the pickup and dropoff dates for the reservation.

b. Relationships

- Vehicles **Has** VehicleClass (vin, classID)
 - Many-to-1 (Required): Every car belongs to a class
- Area **relocates_to** Area (SourceID, TargetID)
 - Attribute : TransferCost
 - Example : Montreal - Toronto : \$500, Alberta - Toronto : \$1000
 - **Note** : The attributes *sourceID* and *targetID* are used solely for identification purposes and do not imply directionality. The *relocates_to* relationship is undirected, and transfer costs are symmetric between the two areas.
- Vehicles **Stored_At** Branch (vin, branchID)
 - Many-to-1 (Required for Vehicles): Every car has a branch for inventory tracking.
- Branch **Belongs_To** Area (branchID, areaID) : Many-to-1 (Required).
- Customer **Makes** Reservation (email, resID)
 - 1-to-Many (Required for Reservation): Bookings must be linked to a customer.
- Reservation **Reserves** RentalPeriod (resID, periodID)
 - 1-to-Many (Required for R): A reservation can reserve multiple rental periods
- Reservation **Fulfills** Agreement (resID, contractID)
 - 1-to-1 (Partial for both): A reservation only becomes an agreement at the moment of pickup.
- Agreement **Assigned_To** Vehicles (contractID, vin)
 - Many-to-1 (Required for Agreement): A legal contract is linked to a car (VIN).
 - **Note** : A vehicle may only be assigned to one active agreement at a time, but *many agreements can be assigned to a vehicle over the operational lifespan*
- BranchManager **Manages** Branch (eID, branchID)
 - 1-to-1 (Required for Manager): One manager is responsible for exactly one branch.
- Employee **Works_At** Branch (eID, branchID)
 - Many-to-1 (Required): All staff are assigned to a branch
- Employee **Reports_To** Employee (eID, supervisorID)
 - Many-to-1 (Recursive): (e.g., Mechanic reports to Manager).
- Agreement **Handled_By** Employee (contractID, eID)
 - Many-to-1 (Required for Agreement): Every transaction must be processed by an employee

3. Application description

a. Overview

The application seeks to efficiently allocate a multi-hub fleet and manpower pool. The goal is to determine an optimal fleet density and labor budget for each Canadian branch. Utilization Rate, Route Profitability, and Market Premium Ranking are **derived at runtime** by the application layer using current

inventory levels, reservation forecasts, and inter-area transfer costs. These values are not stored directly in the database, as they are dynamically recalculated to reflect real-time operational conditions.

b. Runtime calculations

Utilization Rate: The system identifies "Deficit Areas" (not enough cars) and "Surplus Areas" (too many cars) by comparing current branch stock against future *Reservations*.

Route Profitability: The algorithm retrieves the *transferFee* from the *Relocates_To* relationship set. It identifies the "Cheapest Relocation Path" to move a vehicle class (e.g., SUV) from a surplus area (e.g., Alberta) to a deficit area (e.g., Toronto).

Market Premium Ranking: Based on market analysis, population data and consumer trends, the system assigns a higher opportunity cost to vehicles located in Montreal and Toronto.

c. Functions Example

calculateOpportunityCost(areaID, classID): Returns the predicted revenue a vehicle would generate if it remained in its current market.

expectedStaffing(branchID): Returns the estimated number of employees needed to process projected contracts based on incoming *Reservation* spikes.

d. Algorithm Description (Variables to be determined)

Net Migration Gain: The primary output, determining if moving an asset results in a profit after subtracting the *TransferFee* and the **Opportunity Cost** of the source Area.

Optimal Area Fleet Mix: Determining if a regional hub requires a higher density of specific vehicle classes (e.g., Trucks for Western hubs vs. Economy for Metro hubs) based on historical *RentalPeriod* preference data.

Branch Staffing Threshold: Calculating a branch's labor budget based on the ratio of *Agreements* handled relative to the current number of *Employees* assigned to that location.

Inventory Leveling Priority: A ranking of which branches must receive vehicles immediately to avoid "Stock-out Risk."

4. Creativity, Complexity and Teamwork

a. Design choices

The application models fleet rebalancing as an economic optimization problem, incorporating **opportunity cost**, **inter-area transfer fees**, and regional demand in a national multi-hub rental network.

An **is-a** hierarchy is used to distinguish **Branch Managers** from **Employees** solely to enforce the real-world constraint that each branch is uniquely managed, other types of employee abstracted. A weak entity set, **RentalPeriod**, is used to model rental intervals, allowing a single reservation to support extended or multi-segment rental periods.

b. Website Inspirations

The design of our database was inspired by large-scale vehicle rental platforms such as Enterprise Rent-A-Car, particularly with how they manage and coordinate vehicle inventory across areas and branches. Additionally, the bike-sharing platform Bixi served as a good source of inspiration for the

vehicle rebalancing logic, specifically with identifying surplus and deficit locations and relocating vehicles in a cost efficient manner.

c. How we worked as a team

We had one in-person meeting at the start of the project to get to know each other and set up our collaboration workflow, including shared tools for working on the ER diagrams and GitHub for version control in anticipation of future implementation. After that, most of our collaboration took place asynchronously through regular discussions and check-ins. Since this was a multi-part project and everyone had different schedules, we tried to stay flexible and step in for one another when needed. At the same time, each team member's contribution was essential, and the project relied on everyone being involved to come together successfully.

II. Relational Schema

Entities:

Customer (email, name, address, license_expiry)

Reservation (resID, bookedAtTime, email)

email foreign key to Customer

Agreement (contractID, planType, totalCost, eID, vin, resID)

eID foreign key to Employee

vin foreign key to Vehicle

resID foreign key to Reservation

Vehicle(vin, license_plate, mileage, model, status, make, classID, branchID)

classID foreign key to VehicleClass

branchID foreign key to Branch

VehicleClass(classID, className, securityDeposit)

Branch(branchID, address, phone_number, city, areaID, managerID)

areaID foreign key to Area

managerID foreign key to BranchManager

Employee(eID, name, salary, branchID, supervisorID)

branchID foreign key to Branch

supervisorID foreign key to Employee

BranchManager(eID)

eID foreign key to Employee

Area(AreaID, areaName)

Weak Entities:

RentalPeriod(resID, PeriodID, pickupDate, returnDate)

resID foreign key to Reservation

Relations:

Relocation(sourcesAreaID, targetAreaID, fee)

sourcesAreaID foreign key to Area

targetAreaID foreign key to Area